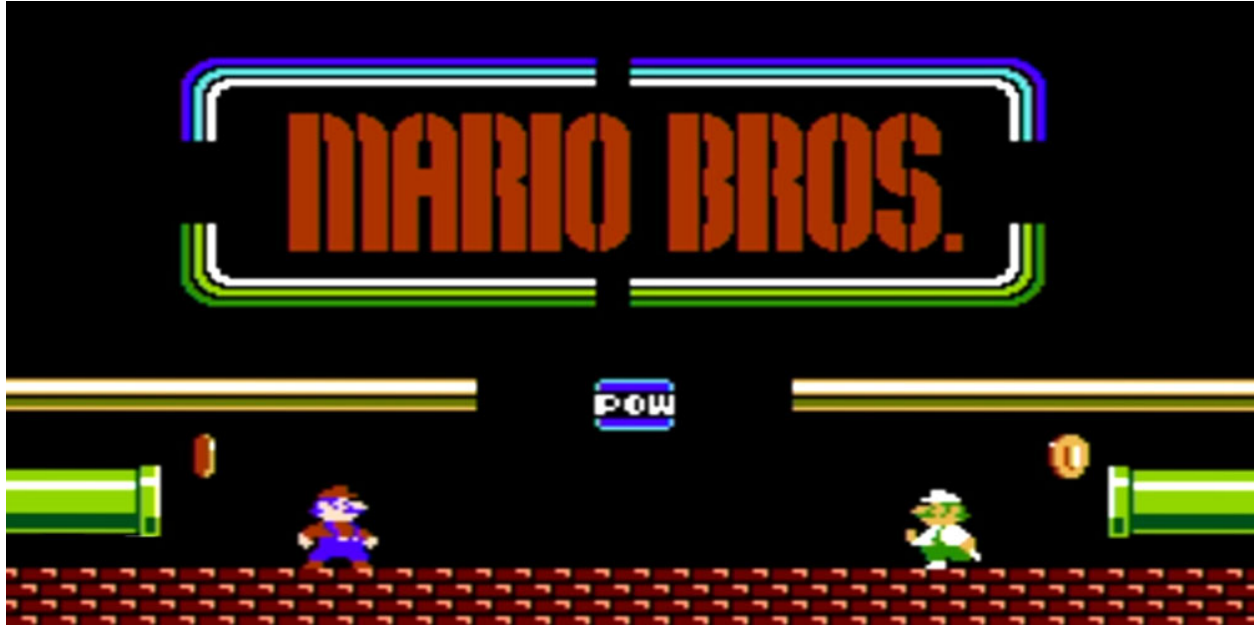


MARIO BROS

FINAL PROJECT PROGRAMMING

JAVIER MOLINA INFANTE, SONSOLES FERNÁNDEZ ESCRIBANO.



"Mario Bros" is an iconic platform video game developed by Nintendo and originally released in 1983 as an arcade game. It was created by Shigeru Miyamoto and is one of the most influential and successful games in the history of video games. It is a platform game in which players control the famous plumbers Mario and Luigi. The game is set in sewers and features a series of platforms where players must face enemies such as turtles and fire crabs.

The main objective of the game is to defeat enemies by hitting them from below and then kicking the resulting shells off the screen before they recover. Players can earn points by defeating enemies and collecting coins.

Although this game is simpler compared to later entries in the Mario series, it laid the groundwork for many of the elements that would become iconic in the franchise, setting the stage for the iconic Super Mario Bros series we know today.

INDEX

Introduction	3
Instructions	4
Summary	5
Mario file	6
Class Mario	
Enemies file	7
Class Enemies	
Class Turtles, Class Flies, Class Crabs	
Platforms file	8
Class Plataformas	
Class Pow	
Class Tuberías	
Class Coins	
Board file	9
Class Board	
Main file	10
Conclusion	10

INTRODUCTION:

For the final project, we have been told to implement a simplified version of the 1983 Mario Bros game.

In our "Mario Bros.", players control Mario. The game takes place in sewers, and the main objective is to defeat the enemy creatures, the turtles or "Shellcreeper", the crabs or "Sidestepper" and the flies or "Fighter Fly", by hitting them from below and then kicking them off the screen. Points can also be earned by collecting coins.

"1983 Mario Bros." is a classic platform game designed for two players, although it can also be played alone. In our case we have opted for the second option. Here are more summarized details on how our version of "Mario Bros" is played:

Characters:

Mario: Players control the famous plumber, Mario.

Scenery:

The game takes place on a series of platforms located in sewers. The platforms are arranged on a single screen, and there is no side scrolling. These are made up of four pipes and several blue blocks, on top of which the characters move.

Enemies:

The main enemies are turtles or "Shellcreeper", crabs or "Sidestepper" and flies or "Fighter Fly". These enemies move across platforms and change direction when they reach an edge or collide with each other.

Aim:

The main objective is to defeat enemies by hitting them from below. When an enemy is hit, it flips over. To completely eliminate an enemy, you must kick the enemies off the screen.

Punctuation:

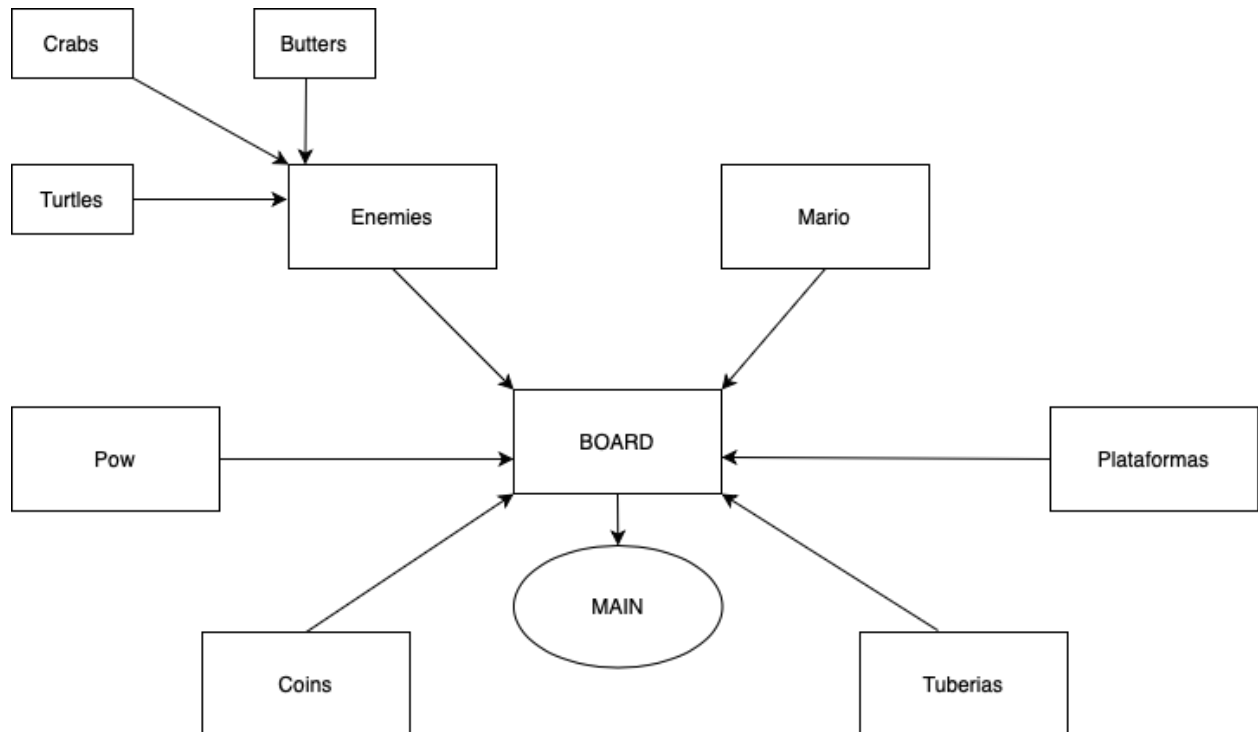
Players can earn points by defeating enemies and collecting coins that appear on some platforms.

INSTRUCTIONS:

The dynamics of the game have already been explained in the introduction. It should only be added that the keys that can be used are those shown here below:



SUMMARY:



Mario, Plataformas, Tuberias, Coins, Pow and Enemies are the classes that are imported in “class Board”, which then is imported to the main file, making our game work.

Turtles, Crabs and Butters are also classes, which base is the Enemies class, as they have inherited from it most of its attributes and methods.

MARIO FILE:

In this file we have created the principal class, Mario class. (The first thing we did was to import pyxel, to have access to this library)

CLASS MARIO:

Attributes:

- "x" and "y": represent the coordinates which show the position of Mario.
- "sprite": tuple with the information of the image of Mario.
- "lives": represents the number of lives that Mario has.
- "dy": represents the speed of Mario along the axis "y", for gravity.
- "air": bool which indicates if Mario is in the air or not.

Methods:

- `__init__`: The initialization method that sets Mario's initial properties, such as his starting position, image, lives and running speed along "y", and whether or not he's in the air.
- `move`: A method that moves Mario in the specified direction ("right" or "left") and with a given length. It also implements logic for changing Mario's image and handles special situations such as teleportation when he crosses screen boundaries.
- `jump`: A method that sets Mario to be in the air and sets his speed accordingly to simulate a jump.

Properties and Setters:

- We have used properties and setters (`@property` and `@setter`) to encapsulate access to the class's attributes.

ENEMIES FILE:

(The first thing we did was to import pyxel, to have access to this library.)

In this file what we have done is a principal class of enemies, and then three more classes which inherit from the previous class using `super().__init__(x, y, direc)`. This means that the enemies classes will inherit the attributes and methods of the Enemies class and can then add or modify them as needed. This three classes are for the “turtles”, “crabs” and “flies”.

CLASS ENEMIES:

Attributes:

- “x” and “y”: represent the coordinates which show the position of the enemies.
- “direc”: represents the speed of Enemies along the axis “x”.
- “sprite”: tuple with the information of the image of the enemies (information common to every enemy).
- “air”: bool which indicates if the enemy is in the air or not.
- “dy”: represents the speed of the enemy along the axis “y”, for gravity.
- “current_platform”: a reference to the current platform where the enemy is.

Methods:

- `__init__`: The initialization method that sets Enemy's initial properties, such as his starting position, image, running speed along “y”, whether or not he's in the air, and the `current_platform`.
- `mover`: A method responsible for moving the enemy in the direction specified by the direction attribute. Implements logic to change the enemy's image based on its direction and Pixels images. This method teleports the enemy from one side of the screen to the other when it crosses the left or right boundaries and adjusts the position x of the enemy according to its direction.

Properties and Setters:

- We have used properties and setters (`@property` and `@setter`) to encapsulate access to the class's attributes.

PLATAFORMAS FILE:

(The first thing we did was import pyxel, to have access to this library.)

Inside this file we have set all types of platforms that create the scenario of our game. It is made up of 5 classes (Plataformas, Floor, Pow, Tuberias and Coins) with similar attributes, which show the position of each element, together with their corresponding sprites. All these 5 classes are initialized with the method `__init__`. Just the coins class presents another method, which is “mover_coins”, used to move the coins along the screen, while its image changes.

As an example we will developed “class Pow” which shows the patro followed by the rest of them

CLASS POW:

Attributes:

- “x” and “y”: represent the coordinates which show the position of the block “pow”.
- “sprite”: tuple with the information of the image of the block, when it is not hit.
- “count_hits”: counter of the number of times it can be hit.
- “hit1_sprite” and “hit2_sprite” : information related to the new image it would have after being hit once and then twice.
- “disappear”: boolean indicating if the block should disappear. It indicates if the block is on the screen.

Methods:

- `__init__`: The initialization method that sets Pow’s initial properties, such as its starting position, sprite, hit counters and whether it has disappeared or not.

BOARD FILE:

We could say that this is the main sprite as it is made up of the class in which all the previously mentioned classes come together.

What we did first was to import all the classes wanted from their corresponding files, together with `pyxel`.

For this class we would just need two parameters, which would be the width and length of the screen.

Attributes:

- `width` and `length(int)`
- `mario`: mario object
- `floor` (list of floor objects)
- `plataformas` (list of plataformas objects)
- `pow` (Pow object)
- `tuberias` (list of tuberias objects)
- `enemies` (list of enemies objects: Flies, Crabs and Turtles)
- `coins` (list of coins objects)
- `current_platform` (platform or none)
- `current_floor` (floor object or none)

Methods:

- `__init__(self, width: int, length: int)`: Initialization method for the Board class.
- `update(self)`: Updates the state of the game, with all the input of the user and the object interactions.
- `collision_floor(self, object)`: Checks if the given object collides with the floor.
- `position_collision_floor(self, object)`: Handles collision with the floor for the given object, that means Mario and the enemies will be above it.
- `enemies_collision_enemies(self)`: Handles collisions between enemies, in such a way that if they collide both change direction.
- `collision_plataformas(self, object)`: Checks if the collision of the given object with the platforms has been formed above or below.
-

-
- `position_collision_plataformas(self, object)`: Knowing how the collision of the objects with the platforms was, it puts the object above or below the platform, considering the gravity and the position of each object .
 - `mario_collision_tuberías(self)`: Handles collision between Mario and pipes.
 - `enemy_collision_tuberias(self)`: Handles collision between enemies and pipes.
 - `mario_collision_pow(self)`: Checks if the collision between Mario and Pow has been from above or below.
 - `position_mario_collision_pow(self)`: Knowing how the collision of Mario with Pow has been, if it has been from above it stays up, but if the collision was from below the pow becomes smaller and at the third time it disappears.
 - `draw(self)`: Draws the game elements using Pixels graphics functions.

MAIN FILE:

This is the file which makes our game work. Here we have just imported from board the class Board adding the parameters needed, which are the dimensions of our screen

CONCLUSION:

As a conclusion of this work we can say that we have learned a lot because as a great wise man said, learning is experience, the rest is just information; and this is what happened to us with our project, when we began to practice everything that in our head was only information has become knowledge. However, this work has been a great challenge, it has been long and hard, but after the solution it has been very rewarding.

That said, and without losing sight of the pride that this work has brought to us, it is true that we would have liked to organize ourselves better to be able to implement the remaining enemies, platforms with more animation, a more appropriate melody for this game or different levels in which the difficulty increased.

But despite these small shortcomings our pride in this work has not decreased, thank you for giving us this opportunity.